05 - Divide and Conquer (QS)

Due Oct 13 at 2pm **Points** 1 **Questions** 3 **Available** until Oct 20 at 2pm
Time Limit 30 Minutes **Allowed Attempts** 10

Take the Quiz Again

Attempt History

	Attempt	Time	Score
LATEST	Attempt 1	6 minutes	0.75 out of 1

ⓘ Correct answers are hidden.

Score for this attempt: **0.75** out of 1
Submitted Oct 19 at 1:32pm
This attempt took 6 minutes.

Partial

Question 1

0.05 / 0.3 pts

Sort the array of integers ⟨8,7,13,6,11,3,1,9, 4⟩ using the quicksort algorithm you learned.

Assume that the partitioning always chooses the first element of the current sub-array as the pivot element.

Give the subarrays computed by the calls to the function **partition(A, p, r)** in order, listing their elements, with the pivot element distinguished by the '+' sign. Separate the elements with commas, don't put any spaces in the text. Example: **4,2,+5,8**

4,7,1,6,3,+8,11,9,13

3,1,+4,6,7

1,+3

+6,7

9,+11,13

How many key comparisons were made during the sorting:

17

Answer 1:

4,7,1,6,3,+8,11,9,13

Answer 2:

3,1,+4,6,7

Answer 3:

1,+3

Answer 4:

+6,7

Answer 5:

9,+11,13

Answer 6:

17

Question 2

0.3 / 0.3 pts

Which statements are true about the **learned (pure) quicksort** algorithm on arrays?

- [x] Partitioning for subarrays with only one element is not performed.
- [ ] The quicksort is a stable sorting algorithm
- [ ] The quicksort is faster than the insertion sort for any input.
- [x] The quicksort is a based on the "divide and conquer" approach.
- [ ] The average time complexity of the quicksort is better, then the time complexity of the merge sort
- [x] The worst-case time complexity of the quicksort is $\Theta(n^2)$
- [ ] It is possible that after partitioning the subarray after the pivot will be empty, but the subarray before the pivot never can be empty.
- [ ] In best case the quicksort sorts the array in linear time.
- [x] During the partitioning, the elements equal with the pivot element can be place both in front and behind the pivot.
- [ ] The quicksort's recursive method is not called for empty subarrays
- [x] Selecting the pivot element and partitioning always takes linear time.
- [x] The best-case time complexity of the quicksort is $\Theta(n \cdot \log n)$

Question 3

0.4 / 0.4 pts

Complete the following pseudo code of stable QuickSort on C2Ls!

Note: In this version of the QuickSort algorithm, the partitioning occurs between the (*p) and (*s) list elements of the C2L. It selects the first item of this partition as the pivot. It checks the other items. Those less than the pivot move between the (*p) element and the pivot.

In the following example, the brackets represent (*p) and (*s), and the linking information is omitted. The pivot is distinguished with a + prefix.
[+3,***2***,7,1,8,2',4,3'] => [2,+3,***7***,1,8,2',4,3'] => [2,+3,7,***1***,8,2',4,3'] => [2,1,+3,7,***8***,2',4,3'] => [2,1,+3,7,8,***2'***,4,3'] => [2,1,2',+3,7,8,***4***,3'] => [2,1,2',+3,7,8,4,***3'***] = [2,1,2',+3,7,8,4,3']

```
QuickSort( H : E2*)
   QS( H, H )

QS( p, s : E2* )
   if  p->next != s and p->next != s->prev
      t := partition( p, s )
      QS( p, t ) ;    QS( t, s )
   end if

partition( p, s : E2* ) : E2*
  [g] ;    [h]
   while [a]
      if [b]
         [c] ;    [d]
         [e];     [f]
      else
         q := q->next
      end if
   end while
   return t
```

[a]	q != s
[b]	p->key < t->key
[c]	r := q
[d]	q := q->next

[e]	unlink(r)
[f]	precede(r, t)
[g]	t := p->next
[h]	q := t->next

Quiz Score: **0.75** out of 1